SALLY

Rust-Native Multi-Model Inference Engine

Engineering Specification & Implementation Blueprint

A modular, memory-aware local inference runtime designed for SALLY and Hermes Agent

Project field	Specification
Project	SALLY Rust-Native Inference Engine
Primary language	Rust for runtime, orchestration, model integration, and local API
Model container	GGUF, subject to exact architecture and tensor-layout compatibility
Target environment	Debian Linux; CPU-first; low-memory hardware
Integration goal	A loopback local API that SALLY and Hermes can call
Schedule	Milestone-driven; no five-day deadline
Document status	Technical plan — implementation and model support must be validated by tests

Prepared for the SALLY development project • 26 October 2026

Document purpose

This document defines the intended architecture, constraints, build sequence, compatibility gates, and integration contract for a Rust-native inference engine for SALLY. It is a build specification, not a claim that every component or target model is already implemented.

The engine should be small, inspectable, testable, and dependable. It should reuse proven compatible compute implementations where that is the sensible engineering choice rather than re-implementing every low-level kernel before a usable prototype exists.

The five-day deadline is explicitly excluded. Work proceeds by measurable milestones and acceptance tests.

Contents

1. Vision and success criteria
2. Scope and non-goals
3. Hardware and feasibility
4. Lessons from AirLLM and Colibrì
5. Proposed architecture
6. GGUF and model compatibility
7. Memory and performance strategy
8. Rust implementation strategy
9. Local API and Hermes integration
10. Security, reliability, and observability
11. Milestone-based build plan
12. Testing and acceptance criteria
13. Model soup: where it fits and where it does not
14. Repository layout
15. Risks and mitigations
16. Reference material

1. Vision and success criteria

Build a Rust-native inference service that can load supported local models, manage memory deliberately, generate text through a stable interface, and integrate with SALLY and Hermes Agent without requiring a heavyweight desktop stack.

Primary objectives

- **Rust-first core:** model loading, lifecycle management, scheduling, memory policy, API server, configuration, logging, and integration code are implemented in Rust.
- **GGUF support:** inspect and load GGUF models safely, but only claim inference support for architectures and tensor types actually implemented or delegated to a compatible backend.
- **Memory awareness:** impose explicit limits on caches, context length, concurrency, and temporary buffers; measure resident memory and disk activity.
- **CPU-first operation:** work without a discrete GPU; optional acceleration can be considered later if supported by the target machine.
- **Stable local interface:** expose health, model listing, and chat-completion endpoints suitable for clients that support an OpenAI-compatible API.
- **Modularity:** separate model-format handling, architecture logic, compute backend, memory policy, API, and client integration.
- **Honest compatibility:** fail clearly on unsupported operations rather than silently producing invalid output.

Definition of success

The first successful release is not defined as “runs every model.” It is a small verified vertical slice: parse a GGUF file, load a known-compatible small model, generate repeatable valid text, expose it through the local API, and pass automated tests under a measured memory budget.

2. Scope and non-goals

In scope

- A Rust command-line service and library crate.
- GGUF metadata and tensor-directory inspection, validation, and diagnostics.
- A model registry with explicit architecture and backend capability checks.
- A bounded memory/cache manager and request scheduler.
- A local HTTP API with health, model listing, chat completions, cancellation, and optional streaming.
- Hermes/SALLY connection documentation and a small smoke-test client.
- Benchmarking and model qualification tools.

Not required for the first version

- Training a new foundation model from scratch.
- A graphical interface, mobile app, or system-wide daemon installer.
- Multi-GPU inference, distributed inference, or cloud hosting.
- Automatic support for every Hugging Face model architecture.
- A custom tensor math library built entirely from first principles before using a working backend.

The engine itself should be Rust-native. If a compatible native compute library is selected, it must be treated as an explicit backend dependency with a documented license, build method, and compatibility boundary. If the project later requires every inference kernel to be pure Rust, that stricter requirement must be treated as a separate architecture decision because it significantly increases implementation scope.

3. Hardware and feasibility

The intended development target is a Debian Linux machine with a low-memory CPU-only configuration: approximately 3.7 GiB of RAM, an Intel Core i3-4000M-class CPU, and Intel HD 4600 integrated graphics. Actual current hardware should be re-measured before tuning.

What this implies

- CPU-only inference is the baseline. Integrated graphics must not be assumed to provide a usable inference backend.
- A small quantized model is the correct first end-to-end target.
- Disk-backed weights or memory mapping can reduce eager allocation, but do not make a large model's full working set disappear.
- Long context lengths and multiple concurrent requests can exhaust memory even when weights fit.
- Disk streaming may make inference extremely slow on a mechanical disk; measure throughput and page faults instead of assuming it will be practical.

Important target-model warning: a 27-billion-parameter model is far beyond the comfortable memory range of a machine with about 3.7 GiB RAM. Even a heavily quantized checkpoint can be many times larger than available physical memory. Streaming may make some designs technically possible, but it does not guarantee useful token speed, sufficient runtime memory, or compatibility with the chosen architecture.

Before selecting a quantization or downloading a large checkpoint, inspect the exact file size, GGUF metadata, architecture identifier, tensor types, context requirements, and runtime support. Do not rely on a model name alone.

4. Lessons from AirLLM and Colibrì

AirLLM and Colibrì are architectural inspirations, not code that should be blindly combined. The engine should reproduce useful principles behind clear Rust interfaces and should verify the actual capabilities and licenses of any upstream project before reusing code.

AirLLM-inspired principle: staged weight loading

- Load or map only the portions needed by the execution plan where the model architecture and compute backend allow it.
- Keep a bounded cache of recently used layers or tensors when reloading them is cheaper than repeatedly reading them from storage.
- Separate model storage from execution so storage policy can evolve without rewriting the API.
- Measure cold-start time, steady-state token rate, disk reads, and peak resident memory.

Colibrì-inspired principle: expert-aware streaming

- Expert streaming is relevant only when the target is a supported mixture-of-experts (MoE) architecture and the runtime understands its routing and tensor layout.
- An MoE-specific optimisation must not be assumed to work for a dense transformer.
- The runtime must understand which experts are selected, when they are needed, and what must remain resident for correct execution.
- Expert caching and prefetching should be added only after correctness is established and traces show that they improve the actual workload.

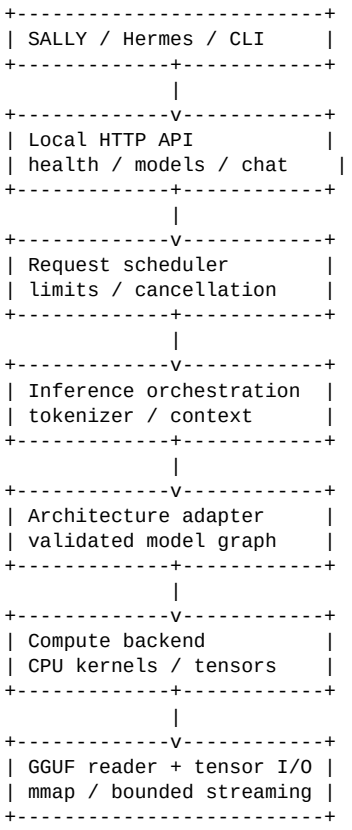
Design rule

Keep generic storage and memory-management interfaces separate from architecture-specific execution. Use an adapter such as a layer-streaming policy or expert-streaming policy only when the model's verified execution graph supports it.

Concern	General engine responsibility	Architecture-specific responsibility
GGUF	Safe parsing, metadata, tensor offsets, validation	Interpret model metadata and map tensor names/shapes correctly
Streaming	I/O, cache limits, eviction, metrics	Define safe load/use/release boundaries
MoE	Storage and cache primitives	Router, expert selection, expert tensor execution
Attention	Memory allocation and backend interface	Correct attention variant, cache semantics, masks, and positions

5. Proposed architecture

Use a layered design with narrow interfaces. The first version should implement only enough architecture support to prove the full path from model file to API response.



Core modules

Module	Responsibility	Initial implementation
config	Parse config, defaults, and command-line overrides	Small TOML or JSON config; validate every limit
gguf	Read header, metadata, tensor descriptors, offsets	Read-only inspector first; reject malformed files safely
model_registry	Map architecture identifiers to capabilities	Explicit allow-list; unsupported is an error
tensor_store	Expose tensor data through backend-neutral handles	Memory-map or bounded file reads where safe
backend	Tensor operations and model execution primitives	Choose a compatible native backend after support audit
architecture	Model graph, positions, attention, MLP/MoE logic	Start with one well-supported small model family
memory	Budgeting, cache, allocation accounting, eviction	Hard caps, metrics, and safe fallback errors
scheduler	Queue, concurrency limits, cancellation	One generation at a time initially
tokenizer	Encode prompts and decode generated tokens	Use the tokenizer implementation compatible with the model
api	HTTP endpoints and response schemas	Loopback-only server and bounded request body
telemetry	Structured logs, timings, memory and token counters	No large tracing stack unless needed

Capability registry

Each supported model entry should declare its architecture identifier, supported tensor types, required operations, tokenizer requirements, context constraints, backend requirements, and known limitations. A model should be marked “supported” only after the relevant integration tests pass.

6. GGUF and model compatibility

GGUF inspection phase

1. Read the magic, version, tensor count, metadata count, and file boundaries with checked arithmetic.
2. Parse key/value metadata without assuming optional fields exist.
3. Record the architecture identifier, dimensions, attention parameters, context-related fields, tokenizer metadata, and quantization-related metadata where present.
4. Read tensor names, shapes, types, and offsets; validate ranges against the file size.
5. Produce a human-readable report and exit with a non-zero status for malformed or unsupported files.

Compatibility gate for the 27B target

The name “Qwen3.8-27B” is not sufficient to establish the actual architecture, checkpoint availability, or GGUF support. Confirm the exact official or trusted model page and exact GGUF file. Then inspect its metadata and compare every required operation against the selected backend and architecture adapter.

- Verify the exact model identifier, publisher, revision, and license.
- Verify that the checkpoint is a GGUF file and inspect its architecture metadata.
- Verify whether it is dense or MoE, and identify attention, recurrent/state-space, or hybrid components if applicable.

- Verify tensor types and quantization support in the backend.
- Verify tokenizer compatibility, special tokens, context handling, positional encoding, and any required recurrent state.
- Run a small, deterministic prompt test and compare expected behavior before enabling API access.

If any required operation is unsupported, the correct result is a documented unsupported-model error or a scoped backend/architecture task—not an unsafe fallback.

7. Memory and performance strategy

Memory budget

Treat memory as a resource budget rather than an optimisation to add at the end. Set separate limits for model weights, runtime scratch buffers, key/value cache or other recurrent state, tokenizer buffers, queued requests, and HTTP bodies.

Resource	Policy
Model weights	Avoid unnecessary copies; choose mapping or streaming based on backend support and measured behavior
Scratch buffers	Reuse allocations where safe; cap temporary workspace
Context/state	Make context length configurable and reject requests exceeding the limit
Concurrency	Start with one active generation; add parallelism only after measuring memory
Cache	Bound by bytes, expose hit/miss/eviction metrics, use safe eviction boundaries
Requests	Bound body size, queue length, output tokens, and generation time
Failure	Return a clear resource-limit error instead of allowing repeated out-of-memory failures

Memory mapping versus streaming

Memory mapping can avoid copying an entire file into an application-owned buffer, but the operating system may still page file-backed data into physical memory. Streaming can control which tensors are read, but may increase latency and I/O. These are different strategies and should be benchmarked independently.

- Measure peak RSS and process memory during model load and generation.
- Measure cold-start and warm-start time separately.
- Record tokens per second after warm-up, not only the first token.
- Track disk read throughput and page faults if available.
- Test with the real target storage device and normal desktop load.
- Do not use swap as a substitute for a feasible model/hardware pairing.

Benchmark matrix

Test	Record	Purpose
Small model, short prompt	load time, first-token latency, tokens/s, peak RSS	Validate basic correctness and baseline
Longer prompt	prompt length, prefill time, state memory	Find context-related pressure
Repeated generation	warm latency, cache hits, memory stability	Detect leaks and repeated I/O costs
Streaming enabled	RSS, disk reads, token rate	Evaluate memory/latency trade-off
Request cancellation	cancellation latency, cleanup, RSS	Ensure resources are released

8. Rust implementation strategy

Principles

- Use stable Rust and keep the dependency set small.
- Keep unsafe code isolated, documented, and covered by tests.
- Use checked arithmetic for all untrusted file offsets and tensor sizes.
- Prefer typed errors over panics for malformed models, I/O failures, and unsupported operations.
- Avoid holding large tensor buffers across unrelated async tasks.
- Separate synchronous CPU-bound inference from HTTP async work so the server remains responsive.
- Pin and audit backend versions; document build flags and optional native dependencies.

Backend selection gate

Before writing custom kernels, compare available Rust-compatible and native CPU backends for the target architecture, GGUF quantization types, licensing, build complexity, memory behavior, and integration maturity. Do not assume a crate supports a model merely because it supports GGUF or a related architecture.

The selected backend must provide the operations needed by the model adapter. If no compatible backend exists, reduce scope to a supported model first and treat target architecture support as a separate engineering milestone.

9. Local API and Hermes integration

Proposed API

Endpoint	Purpose	Initial behavior
GET /health	Liveness and basic status	Return service status and loaded-model state
GET /v1/models	List models available to clients	Expose only loaded or explicitly configured models
POST /v1/chat/completions	Generate assistant responses	Accept messages, model, max tokens, temperature; validate limits
POST /admin/load (optional)	Load a configured model	Disabled by default or restricted to local administration
POST /admin/unload (optional)	Release model resources	Wait for active request to finish or cancel safely

The API should adopt the smallest useful subset of the OpenAI chat-completions schema needed by the client. Document unsupported fields rather than pretending to implement them. Streaming responses can be added after non-streaming generation is stable.

Hermes connection sequence

1. Start SALLY inference service bound to 127.0.0.1 by default.
2. Call GET /health and verify that the service is ready.
3. Call GET /v1/models to discover the model identifier exposed by the engine.
4. Configure Hermes to use the local OpenAI-compatible endpoint and the discovered model identifier, following the installed Hermes version's provider configuration.
5. Send a short test prompt and confirm that the response is produced by the local model.
6. Test failure behavior: engine stopped, model unsupported, memory limit reached, and request cancelled.

```
# Conceptual endpoint only; use the exact setting names
# supported by the installed Hermes version.
BASE_URL=http://127.0.0.1:<configured-port>/v1
```

```
MODEL=<model-id-returned-by-SALLY>
```

Do not expose the API to the LAN or internet by default. If remote access is later required, add authentication, explicit bind configuration, and firewall guidance before enabling it.

10. Security, reliability, and observability

- Treat model files and metadata as untrusted input; validate all sizes, offsets, and counts before allocation or reading.
- Keep the service loopback-only by default and avoid shell execution from model content or request text.
- Set request body, prompt length, output token, queue, and time limits.
- Return structured error codes for unsupported architecture, invalid model, resource exhaustion, cancellation, and internal failure.
- Use structured logs without dumping full prompts by default.
- Track load duration, prompt tokens, generated tokens, first-token latency, tokens per second, peak memory, cache statistics, and error counts.
- Ensure cancellation releases request-local state and does not corrupt the shared model.
- Add a graceful shutdown path that stops accepting requests and cleans up resources.

11. Milestone-based build plan

There is no fixed five-day deadline. Complete each milestone when its acceptance criteria pass. Keep the initial vertical slice small, then add memory optimisations and target-model support based on evidence.

Milestone	Deliverable	Exit criteria
M0 — Audit	Review SALLY repo, Rust workspace, current dependencies, Hermes config needs	Document existing code to reuse and avoid duplicate components
M1 — GGUF inspector	CLI reads metadata and tensor descriptors	Valid sample parses; malformed files fail safely
M2 — Backend decision	Backend capability matrix and build proof	A small compatible model can load with known operations
M3 — First generation	CLI loads small model and generates text	Correct tokenizer and deterministic smoke test pass
M4 — Memory controls	Budget, context caps, metrics, cache policy	No unbounded queue/cache; memory tests documented
M5 — HTTP API	Health, models, non-streaming chat completion	Schema tests and error paths pass
M6 — Hermes integration	Local endpoint configured and smoke-tested	Hermes can send a prompt and receive output
M7 — Streaming and cancellation	Optional incremental output and cleanup	Cancellation test passes without resource corruption
M8 — Target model qualification	Exact 27B model metadata and capability audit	Supported only if every required operation passes
M9 — Performance tuning	Measured benchmarks and documented trade-offs	Changes improve a stated metric without regressions

12. Testing and acceptance criteria

Required test categories

- **Unit tests:** GGUF header parsing, metadata values, tensor offsets, configuration validation, token accounting, and error mapping.

- **Malformed-file tests:** truncated files, impossible counts, overflow offsets, invalid tensor dimensions, and unsupported versions.
- **Integration tests:** load a small known-compatible model, tokenize, generate, and decode.
- **API tests:** health, model list, valid request, invalid JSON, unsupported model, excessive context, and cancellation.
- **Resource tests:** repeated requests, queue limit, bounded cache, model unload, and graceful shutdown.
- **Client test:** one end-to-end request through Hermes using the local endpoint.

Release checklist

- Build succeeds on the intended Debian target from a clean checkout.
- Documented commands reproduce the build and tests.
- The supported model and quantization are named exactly.
- Peak RSS, load time, first-token latency, and generation speed are recorded.
- No test claims the 27B target is supported unless its architecture adapter and backend operations have passed tests.
- The API is loopback-only unless an explicit secure remote-access configuration is enabled.
- Known limitations and license obligations are documented.

13. Model soup: where it fits and where it does not

Model soup generally means merging compatible model weights, often by averaging checkpoints or combining fine-tunes under specific conditions. It is a model-development technique, not an inference-memory optimisation.

What it can do

- Potentially combine compatible fine-tuned checkpoints or improve a selected quality metric when the merge assumptions hold.
- Create a new checkpoint that can later be quantized and evaluated as a separate model artifact.

What it cannot be assumed to do

- It does not automatically reduce parameter count or memory requirements.
- It does not automatically make an unsupported architecture work in the runtime.
- It does not replace quantization, model distillation, pruning, or a memory-aware execution strategy.
- It may reduce quality or fail when checkpoints differ in architecture, tensor layout, tokenizer, or parameter alignment.

Recommendation: exclude model soup from the inference engine’s first implementation. If model merging is explored later, keep it as a separate offline experiment with provenance, license review, validation benchmarks, and an independently versioned output checkpoint.

14. Repository layout

```
SALLY/
■■■ native/
■   ■■■ sally-inference/
■       ■■■ Cargo.toml
■       ■■■ src/
■       ■   ■■■ main.rs              # CLI/service entry point
■       ■   ■■■ lib.rs
■       ■   ■■■ config.rs
■       ■   ■■■ error.rs
■       ■   ■■■ gguf/
■       ■   ■   ■■■ mod.rs
■       ■   ■   ■■■ reader.rs
■       ■   ■   ■■■ metadata.rs
■       ■   ■   ■■■ tensor_index.rs
■       ■   ■■■ model_registry.rs
■       ■   ■■■ tensor_store.rs
■       ■   ■■■ backend/
■       ■   ■■■ architecture/
■       ■   ■■■ memory.rs
■       ■   ■■■ scheduler.rs
■       ■   ■■■ tokenizer.rs
■       ■   ■■■ api/
■       ■   ■■■ telemetry.rs
■       ■■■ tests/
■       ■■■ benches/
■■■ docs/
■   ■■■ inference-engine-specification.pdf
■   ■■■ backend-capability-matrix.md
■   ■■■ model-qualification.md
■■■ scripts/
    ■■■ inspect-gguf
    ■■■ smoke-test-local-api
```

This is a proposed layout, not a demand to move or rewrite existing SALLY components without first auditing the current repository. Preserve working code and integrate incrementally.

15. Risks and mitigations

Risk	Impact	Mitigation
Target model name or metadata is inaccurate	Wrong architecture assumptions and wasted work	Verify exact checkpoint and inspect GGUF before implementing support
No backend supports required operations	Target model cannot run	Start with a known-supported small model; isolate architecture work
27B model exceeds RAM	Thrashing, extreme latency, or failure	Use a small model for development; measure streaming; document hardware limits
Disk streaming is too slow	Unusable token rate	Benchmark early; choose mmap/cache strategy based on storage and backend
Custom kernels expand scope	Long delay and correctness bugs	Reuse compatible kernels behind a Rust interface where acceptable
Model parser bug	Crash, invalid reads, or corrupted results	Checked arithmetic, strict validation, fuzzing and malformed-file tests
Hermes API mismatch	Client cannot connect	Use exact installed Hermes provider documentation and run an end-to-end test
Remote API exposed without protection	Unauthorised use or data exposure	Loopback-only default; explicit secure configuration for remote access

16. Reference material

Use these upstream resources to verify implementation details, supported architectures, current APIs, and license terms. The presence of a reference does not imply that it supports the exact target checkpoint.

GGUF specification and llama.cpp: https://github.com/ggml-org/llama.cpp

GGUF documentation in llama.cpp: https://github.com/ggml-org/llama.cpp/tree/master/gguf-py

AirLLM project: https://github.com/lyogavin/Anima

Colibrì project: https://github.com/Colibr-ai/Colibri

Rust language: https://www.rust-lang.org/

Rust package registry: https://crates.io/

Hermes Agent repository: https://github.com/NousResearch/hermes-agent

Before using any upstream code or model weights, review the project's current repository, license, maintenance state, and compatibility documentation. The AirLLM and Colibrì URLs above are starting points for research; verify the exact projects and their current names/locations before adopting code.

Final engineering direction

Build a small, verified Rust inference service first. Make GGUF inspection and backend capability reporting reliable. Prove generation with a supported small model, add memory limits and measurements, then connect the local API to Hermes. Only after these gates pass should the exact 27B target be qualified. Keep the architecture modular enough to add layer- or expert-streaming policies without claiming those strategies work for every model.

The guiding rule for SALLY is simple: correctness first, measurable resource use second, optimisation third. Model support is earned by tests, not inferred from a model name.